

1. S-DES Key Generation (K1 & K2)

```
# S-DES Key Generation

P10 = [3, 5, 2, 7, 4, 10, 1, 9, 8, 6]
P8  = [6, 3, 7, 4, 8, 5, 10, 9]

def permute(key, table):
    return ''.join(key[i - 1] for i in table)

def left_shift(bits, shifts):
    return bits[shifts:] + bits[:shifts]

def generate_keys(key):
    print("Original Key:", key)

    key = permute(key, P10)
    print("After P10:", key)

    left = key[:5]
    right = key[5:]

    # LS-1
    left = left_shift(left, 1)
    right = left_shift(right, 1)
    combined = left + right

    k1 = permute(combined, P8)
    print("K1:", k1)

    # LS-2
    left = left_shift(left, 2)
    right = left_shift(right, 2)
    combined = left + right

    k2 = permute(combined, P8)
    print("K2:", k2)

    return k1, k2

key_input = input("Enter 10-bit binary key: ")
generate_keys(key_input)
```

2. DES Encryption & Decryption

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
```

```

def des_encrypt(plaintext, key):
    cipher = DES.new(key, DES.MODE_ECB)
    padded_text = pad(plaintext.encode(), DES.block_size)
    ciphertext = cipher.encrypt(padded_text)
    return ciphertext

def des_decrypt(ciphertext, key):
    cipher = DES.new(key, DES.MODE_ECB)
    decrypted = unpad(cipher.decrypt(ciphertext),
DES.block_size)
    return decrypted.decode()

key = b'8bytekey' # must be 8 bytes

plaintext = input("Enter message: ")

cipher = des_encrypt(plaintext, key)
print("Encrypted:", cipher)

decrypted = des_decrypt(cipher, key)
print("Decrypted:", decrypted)

```

3. AES SubBytes & ShiftRows (4x4 Matrix)

```

# Example AES state matrix
state = [
    [0x19, 0xa0, 0x9a, 0xe9],
    [0x3d, 0xf4, 0xc6, 0xf8],
    [0xe3, 0xe2, 0x8d, 0x48],
    [0xbe, 0x2b, 0x2a, 0x08]
]

# AES S-Box (partial for demo, use full in exam if needed)
sbox = [
    [0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30,
0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76],
    # (complete S-box required in real implementation)
]

def print_state(s):
    for row in s:
        print([hex(x) for x in row])
    print()

def sub_bytes(state):
    for i in range(4):
        for j in range(4):
            row = state[i][j] >> 4
            col = state[i][j] & 0x0F

```

```

        state[i][j] = sbox[row][col]
    return state

def shift_rows(state):
    state[1] = state[1][1:] + state[1][:1]
    state[2] = state[2][2:] + state[2][:2]
    state[3] = state[3][3:] + state[3][:3]
    return state

print("Original:")
print_state(state)

state = sub_bytes(state)
print("After SubBytes:")
print_state(state)

state = shift_rows(state)
print("After ShiftRows:")
print_state(state)

```

4. AES Encryption & Decryption

```

from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

key = b'This is a key123' # 16 bytes

def encrypt(msg):
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(msg.encode(), AES.block_size))
    return cipher.iv, ciphertext

def decrypt(iv, ciphertext):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return unpad(cipher.decrypt(ciphertext),
AES.block_size).decode()

msg = input("Enter message: ")

iv, encrypted = encrypt(msg)
print("Encrypted:", encrypted)

decrypted = decrypt(iv, encrypted)
print("Decrypted:", decrypted)

```

5. Diffie-Hellman Key Exchange

```

# Diffie-Hellman

```

```

p = 23
g = 5

a = 6    # Private key of A
b = 15   # Private key of B

A = pow(g, a, p)
B = pow(g, b, p)

print("Public key of A:", A)
print("Public key of B:", B)

# Secret keys
secret_A = pow(B, a, p)
secret_B = pow(A, b, p)

print("Secret key (A):", secret_A)
print("Secret key (B):", secret_B)

```

6. SHA-1 Hash

```

import hashlib

message = input("Enter message: ")

msg_bytes = message.encode()
sha1 = hashlib.sha1(msg_bytes).hexdigest()

print("Message:", message)
print("SHA-1 Hash:", sha1)

```

7. MD5 Hash

```

import hashlib

text = input("Enter message: ")

data = text.encode()
digest = hashlib.md5(data).hexdigest()

print("Message:", text)
print("MD5 Digest:", digest)

```

1. Rail Fence Cipher (Encryption & Decryption)

```
# Rail Fence Cipher

def rail_fence_encrypt(text, depth=2):
    text = text.replace(" ", "").lower()
    rails = ['' for _ in range(depth)]

    row = 0
    direction = 1

    for char in text:
        rails[row] += char
        row += direction

        if row == 0 or row == depth - 1:
            direction *= -1

    return ''.join(rails).upper()

def rail_fence_decrypt(cipher, depth=2):
    n = len(cipher)

    # Create pattern
    pattern = [['\n' for _ in range(n)] for _ in range(depth)]

    row, direction = 0, 1
    for col in range(n):
        pattern[row][col] = '*'
        row += direction

    if row == 0 or row == depth - 1:
        direction *= -1

    # Fill pattern with ciphertext
    index = 0
    for i in range(depth):
        for j in range(n):
            if pattern[i][j] == '*' and index < n:
                pattern[i][j] = cipher[index]
                index += 1

    # Read plaintext
    result = ""
    row, direction = 0, 1

    for col in range(n):
        result += pattern[row][col]
        row += direction
```

```

        if row == 0 or row == depth - 1:
            direction *= -1

    return result.upper()

# Example
text = "meet me after the toga party"
cipher = rail_fence_encrypt(text, 2)

print("Cipher Text:", cipher)

plain = rail_fence_decrypt(cipher, 2)
print("Decrypted Text:", plain)

```

2. Row Column (Columnar Transposition Cipher)

```

# Columnar Transposition Cipher

def encrypt_columnar(plaintext, key):
    plaintext = plaintext.replace(" ", "").lower()
    cols = len(key)
    rows = (len(plaintext) + cols - 1) // cols

    # Fill matrix row-wise
    matrix = [['x' for _ in range(cols)] for _ in range(rows)]

    k = 0
    for i in range(rows):
        for j in range(cols):
            if k < len(plaintext):
                matrix[i][j] = plaintext[k]
                k += 1

    # Read column-wise based on key
    ciphertext = ""
    for num in range(1, cols + 1):
        col = key.index(str(num))
        for row in range(rows):
            ciphertext += matrix[row][col]

    return ciphertext.upper()

def decrypt_columnar(ciphertext, key):
    ciphertext = ciphertext.lower()
    cols = len(key)
    rows = len(ciphertext) // cols

```

```

matrix = [['' for _ in range(cols)] for _ in range(rows)]

k = 0
for num in range(1, cols + 1):
    col = key.index(str(num))
    for row in range(rows):
        matrix[row][col] = ciphertext[k]
        k += 1

plaintext = ""
for i in range(rows):
    for j in range(cols):
        plaintext += matrix[i][j]

return plaintext.rstrip('x').upper()

# Example
key = "4312567"
text = "attack postponed until two am"

cipher = encrypt_columnar(text, key)
print("Cipher Text:", cipher)

plain = decrypt_columnar(cipher, key)
print("Decrypted Text:", plain)

```